XJTLU Entrepreneur College (Taicang) Cover Sheet

Module code and Title	Introduction to Neural Networks
School Title	School of Artificial Intelligence and Advanced Computing
Assignment Title	Coursework
Submission Deadline	24:00 China time (UTC+8 Beijing) on Sunday 6th April 2025
Final Word Count	NA
If you agree to let the university use your work anonymously for teaching and learning purposes, please type **"yes"** here.	

I certify that I have read and understood the University's Policy for dealing with Plagiarism, Collusion and the Fabrication of Data (available on Learning Mall Online). With reference to this policy I certify that:

- My work does not contain any instances of plagiarism and/or collusion. My work does not contain any fabricated data.

By uploading my assignment onto Learning Mall Online, I formally declare that all of the above information is true to the best of my knowledge and belief.

Scoring – For Tutor Use	
Student ID	**2363570**

Stage of Marking	**Marker Code**	**Learning Outcomes Achieved (F/P/M/D) (please modify as appropriate)**			**Final Score**
		A	**B**	**C**	
1st Marker – red pen					
Moderation – green pen	**IM Initials**	The original mark has been accepted by the moderator (please circle as appropriate):			Y / N
		Data entry and score calculation have been checked by another tutor (please circle):			Y
2nd Marker if needed – green pen					

For Academic Office Use			**Possible Academic Infringement (please tick as appropriate)**	
Date Received	**Days late**	**Late Penalty**	☐ **Category A**	Total Academic Infringement Penalty (A,B, C, D, E, Please modify where necessary) _____
			☐ **Category B**	
			☐ **Category C**	
			☐ **Category D**	
			☐ **Category E**	

Step 1

Firstly, I chose ‘backpack’ as my tenth class, because the shapes of different types of backpacks are quite similar.

Secondly, I wrote a crawler programme (import icrawler) to download 'backpack' images from Bing and did some filter operations.

After getting 103 images, I manually annotated 83 of them through Label Studio, I used ‘category=’ to adjust the order of classes.

I chose 20 of the 83 annotated images with their labels and put them into the ‘val’ folders respectively.

Then, I put the rest 63 images with their labels into the ‘train’ folders.

I also put the 20 unannotated images into the ‘images_test’ folder.

Thus, I established my train, validation and test sets.

Step 2

As I mentioned above, my test set now owns 200 (given) +20 (added by me) =220 unannotated images.

My train set owns 1800 (given) + 63 (added) = 1863 annotated images and corresponding labels.

My validation set owns 180 (given) + 20 (added) = 200 annotated images and corresponding labels.

The programme that randomly plots 5 images in the training set with their corresponding labels is in the notebook.

To make sure the size of the submitted zip will not be too big, I will only conclude the images and labels I added, without any given resource. (It means that the train folder will only conclude 63 images of backpack, and val and image_test folders are similar.)

Step 3

I just downloaded ‘yolo11.yaml’ from GitHub of ultralytics, I changed the value of ‘nc’ (number of classes) to 10. The code of object detection model is shown in the ‘yolo11_10class.yaml’ file in the path ‘cfg/models/yolo11_model_10class.yaml’, I have added comments in this file.

I will calculate the total parameters of YOLO11n model, which is used in this project. The parameter calculation details, and results are as follows:

Layer0: 464 (Conv) arguments: [3, 16, 3, 2]

Layer1: 4672 (Conv) arguments: [16, 32, 3, 2]

Layer2: 6640 (C3k2) arguments: [32, 64, 1, False, 0.25]

Layer3: 36992 (Conv) arguments: [64, 64, 3, 2]

Layer4: 26080 (C3k2) arguments: [64, 128, 1, False, 0.25]

Layer5: 147712 (Conv) arguments: [128, 128, 3, 2]

Layer6: 87040 (C3k2) arguments: [128, 128, 1, True]

Layer7: 295424 (Conv) arguments: [128, 256, 3, 2]

Layer8: 346112 (C3k2) arguments: [256, 256, 1, True]

Layer9: 164608 (SPPF) arguments: [256, 256, 5]

Layer10: 249728 (C2PSA) arguments: [256, 256, 1]

Layer11: 0 (Upsample) arguments: [None, 2, 'nearest']

Layer12: 0 (Concat) arguments: [1]

Layer13: 111296 (C3k2) arguments: [384, 128, 1, False]

Layer14: 0 (Upsample) arguments: [None, 2, 'nearest']

Layer15: 0 (Concat) arguments: [1]

Layer16: 32096 (C3k2) arguments: [256, 64, 1, False]

Layer17: 36992 (Conv) arguments: [64, 64, 3, 2]

Layer18: 0 (Concat) arguments: [1]

Layer19: 86720 (C3k2) arguments: [192, 128, 1, False]

Layer20: 147712 (Conv) arguments: [128, 128, 3, 2]

Layer21: 0 (Concat) arguments: [1]

Layer22: 378880 (C3k2) arguments: [384, 256, 1, True]

Layer23: 432622 (Detect) arguments: [10, [64, 128, 256]]

Total number of paraments:

464+4,672+6,640+36,992+26,080+147,712+87,040+295,424+346,112+164,608+249,728+0+0+111,296+0+0+32,096+36,992+0+86,720+147,712+0+378,880+432,622 = 2,591,790

Step 4

I will submit a .py file which includes my training code.

And I also conclude my training process in the notebook. I learnt from Professor Huang that the requirement ‘Not to use notebook to train your model’ is set to help some students who don’t have a good computer, but my computer performs well as well as I set appropriate hyperparameters for the training, so it can make the thing done. Thus, I just used the notebook.

However, I will also submit a train.py file.

The training process is too long to take a screenshot. I will show the final part as below. As for the complete training process, it can be seen in the notebook

Figure 1. Final part of training process

```
35 epochs completed in 0.162 hours.
Optimizer stripped from runs\detect\train\weights\last.pt, 5.5MB
Optimizer stripped from runs\detect\train\weights\best.pt, 5.5MB

Validating runs\detect\train\weights\best.pt...
Ultralytics 8.3.94  Python-3.10.6 torch-2.6.0+cu126 CUDA:0 (NVIDIA GeForce RTX 4070 Laptop GPU, 8188MiB)
YOLO11_model_10class summary (fused): 100 layers, 2,584,102 parameters, 0 gradients, 6.3 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100%|██████████| 7/7 [00:01<00:00,  5.82it/s]
                   all        200        518      0.336      0.296      0.294      0.186
                person         20         90      0.357      0.289      0.266      0.133
                   cat         20         23      0.364      0.391      0.418      0.212
                   dog         20         22      0.429      0.364      0.379      0.235
                bottle         20         55      0.217      0.181      0.132     0.0591
                banana         20         47       0.18      0.128      0.107      0.063
                orange         20        106      0.559      0.538      0.492       0.36
                 chair         20         61     0.0878     0.0164     0.0161    0.00662
                laptop         20         25      0.477        0.4      0.459      0.356
                  book         20         67     0.0264     0.0149      0.013    0.00682
              backpack         20         22      0.666      0.636      0.655      0.427
Speed: 0.2ms preprocess, 1.3ms inference, 0.0ms loss, 0.8ms postprocess per image
Results saved to runs\detect\train
```

As for hyperparameters, I set epoch number as 50, because training beyond this point will increase time cost but get limited improvement.

I use GPU to train my model by setting the value of device to ‘0’ to improve training speed.

I set batch size to 16. I tried to improve the batch size to 32. However, the code returned an error: ‘torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 150.00 MiB.’ This indicates that my GPU was fully utilized with batch=32. Therefore, I reduced the batch size to 16, which turned out to be the maximum size my hardware could handle.

I set an input image size of 640×640, which is a default resolution balancing speed and accuracy.

I chose Adam as an optimizer, because it's faster and more friendly to small data sets than SGD. For small datasets, SGD is more likely to overfit.

Then I set the value of ‘lr0’ (initial learning rate) as 0.001, as Adam is an adaptive optimizer which can adjust learning rates for each parameter, and a small initial learning rate can ensure stable training.

Additionally, I set the value of ‘patience’ as 10 to allow early stopping if the model can’t witness improvement after 10 epochs. In fact, my training stopped early when the number of epochs is 35.

To visualize training and validation performance, I created a function plot_training_performance to plot the train loss, val loss, train acc, and val acc. The code, comments and the graphs are shown in the notebook.

Step 5

Firstly, I write a function ‘display_predictions’ to display the predictions of the randomly selected test images and visualize the result.

Then, I calculate the mAP for training and validation images.

Lastly, I calculate the confusion matrix, I will show them as below.

Figure 2. Normalized Confusion Matrix

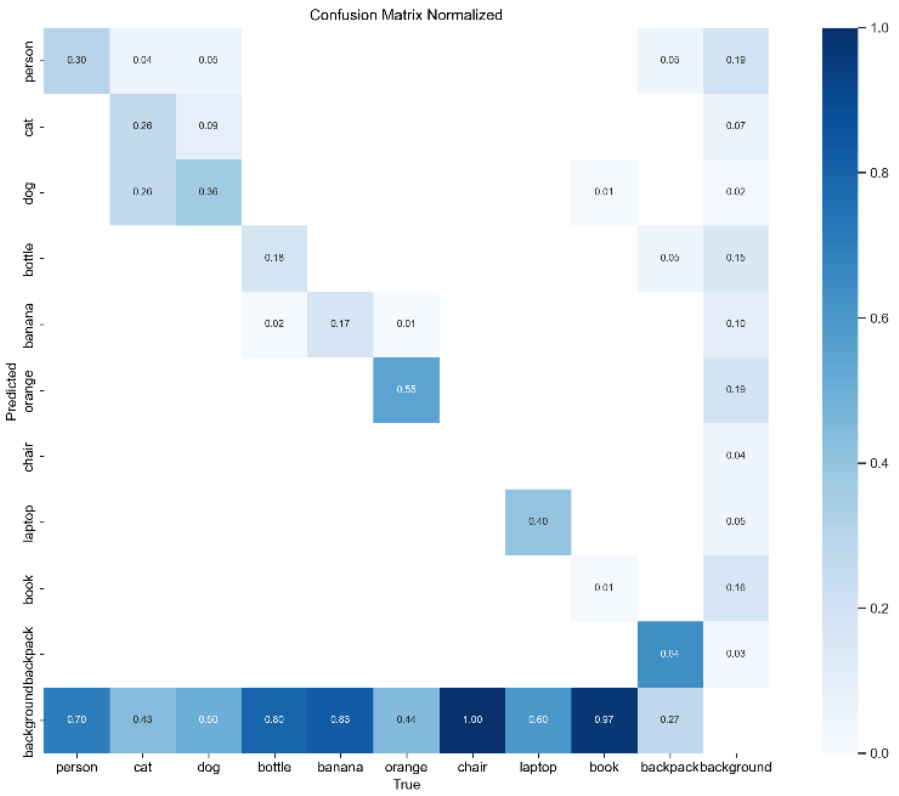
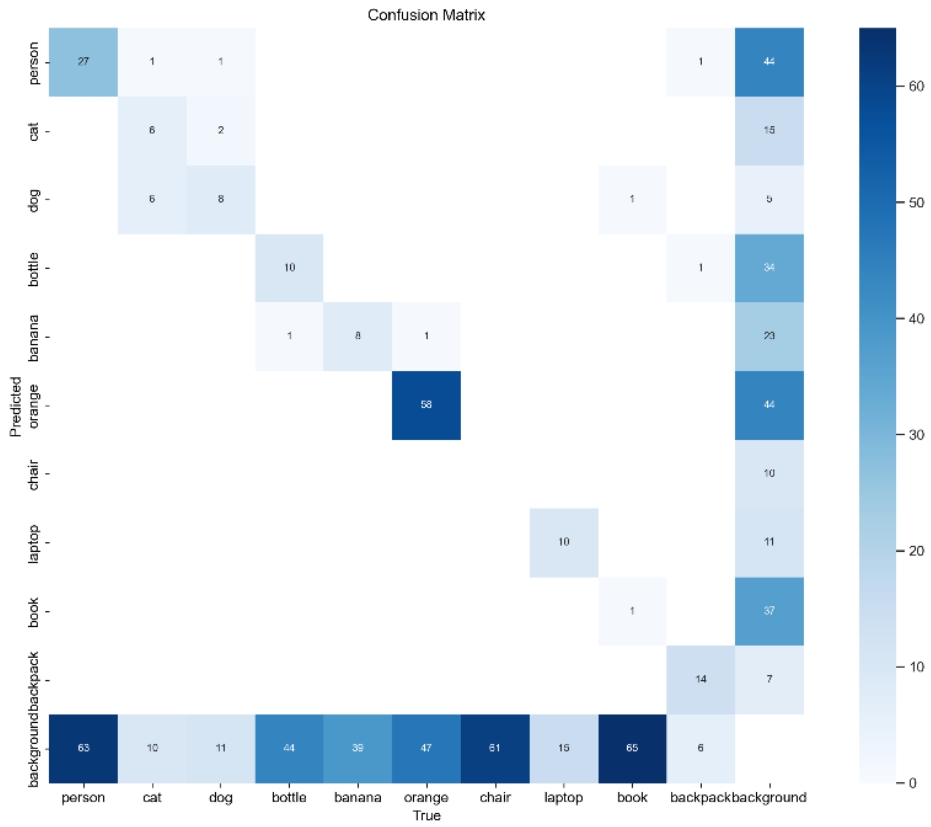


Figure 3. Confusion Matrix



All the code and results mentioned above are contained in the notebook.